

Digital Logic

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Contents

1	Number Representation	4
1.1	Range of n-bit 2's Complement Numbers	4
1.2	Range of n-bit 1's Complement Numbers	4
2	Boolean Algebra and Logic Minimization	5
2.1	Implicants	5
2.1.1	Implicant	5
2.1.2	Prime Implicant	5
2.1.3	Essential Prime Implicant	5
2.2	Finite State Machine to Calculate 2's Complement	6
3	Sequential Logic: Flip-Flops	7
3.1	SR (Set-Reset) Flip-Flop	7
3.2	T (Toggle) Flip-Flop	7
3.3	D (Delay) Flip-Flop	7
3.4	JK Flip-Flop	8
3.5	Characteristic Equation	8
4	Advanced Sequential Logic	9
4.1	Clocked Flip-Flops with Clear and Reset Inputs	9
4.2	Logic Levels	9
4.2.1	Positive Logic	9
4.2.2	Negative Logic	9
4.3	Minimum Number of Gates Required	9
4.4	Finite State Machines	9
4.4.1	Moore Machine	9
4.4.2	Mealy Machine	10
5	Combinational Logic Components	11
5.1	Multiplexers	11
5.2	Decoders	11
5.3	Encoders	12
5.4	Canonical Logic Forms	12
5.4.1	Half Adder	12
5.4.2	Full Adder	12
5.5	Look-Ahead Carry Adder	12

6	Arithmetic Circuits	13
6.1	Half Subtractor	13
6.2	Full Subtractor	13
7	State Machine Minimization	14
7.1	Reduction of State Tables	14
7.1.1	Example	14
8	Code Conversion	15
8.1	Binary Code to Gray Code Conversion	15
8.2	Gray Code to Binary Code Conversion	15

Chapter 1

Number Representation

1.1 Range of n-bit 2's Complement Numbers

The range of an n-bit 2's complement number is:

$$-2^{(n-1)} \text{ to } (2^{(n-1)} - 1)$$

1.2 Range of n-bit 1's Complement Numbers

The range of an n-bit 1's complement number is:

$$-(2^{(n-1)} - 1) \text{ to } (2^{(n-1)} - 1)$$

Chapter 2

Boolean Algebra and Logic Minimization

2.1 Implicants

2.1.1 Implicant

A normal product term that implies a Boolean function Y is called an **implicant**.

$$Y = AB + ABC\bar{C} + \bar{B}C$$

All terms AB , $ABC\bar{C}$, and $\bar{B}C$ are implicants of Y .

2.1.2 Prime Implicant

An implicant of Y is called a **prime implicant** if any variable is removed from the implicant, the resulting term does not imply Y .

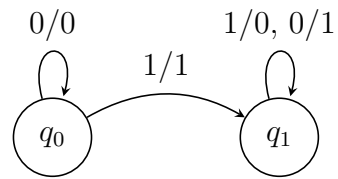
Example: Consider $Y = AB + \bar{B}C + ABC$

- This is not a prime implicant because if we remove variable A from ABC , we get $\bar{B}C$, which still implies Y .
- Prime implicant: $AB + \bar{B}C$

2.1.3 Essential Prime Implicant

If a minterm is covered only by a prime implicant, then that prime implicant is called an **essential prime implicant**.

2.2 Finite State Machine to Calculate 2's Complement



Input is from ~~least~~ significant bit

Chapter 3

Sequential Logic: Flip-Flops

3.1 SR (Set-Reset) Flip-Flop

$S(t)$	$R(t)$	$Q(t)$	$Q(t+1)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	Inputs not allowed
1	1	1	

Characteristic Equation:

$$Q^+ = S + \bar{R}Q$$

where $SR = 0$ (condition for the type of flip-flop to be employed).

3.2 T (Toggle) Flip-Flop

T	Q	Q^+
0	0	0
0	1	1
1	0	1
1	1	0

Characteristic Equation:

$$Q^+ = T\bar{Q} + \bar{T}Q = T \oplus Q$$

3.3 D (Delay) Flip-Flop

D	Q	Q^+
0	0	0
0	1	0
1	0	1
1	1	1

Characteristic Equation:

$$Q^+ = D$$

3.4 JK Flip-Flop

J	K	Q	Q^+
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Characteristic Equation:

$$Q^+ = J\bar{Q} + \bar{K}Q = Q\bar{K} + \bar{Q}J$$

3.5 Characteristic Equation

A characteristic equation is an equation that expresses the next state of a flip-flop in terms of its present state and inputs.

Chapter 4

Advanced Sequential Logic

4.1 Clocked Flip-Flops with Clear and Reset Inputs

- If a 0 is applied to the **clear** input, the flip-flop will reset to $Q = 0$.
- If a 0 is applied to the **preset** input, the flip-flop will set to $Q = 1$.
- These inputs override the clock and J, K inputs.

4.2 Logic Levels

4.2.1 Positive Logic

- $+V$ represents a logic 1
- 0 volts represents a logic 0

4.2.2 Negative Logic

- $+V$ represents a logic 0
- 0 volts represents a logic 1

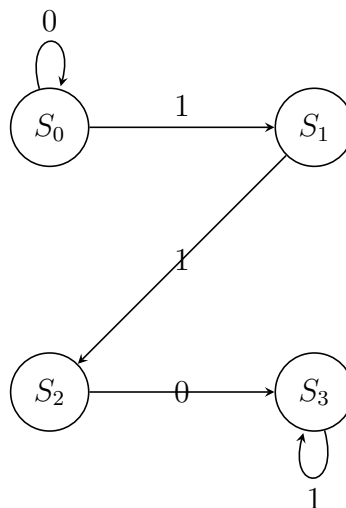
4.3 Minimum Number of Gates Required

To find a minimum solution for an expression, one must find both the network with the AND-gate output and the one with the OR-gate output.

4.4 Finite State Machines

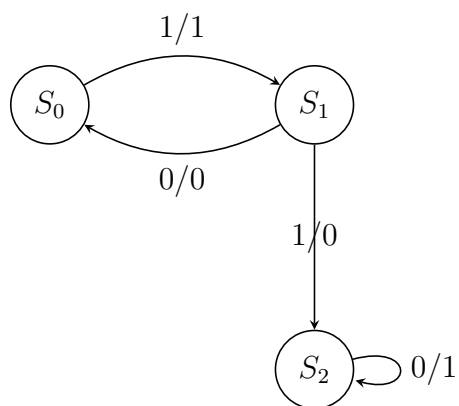
4.4.1 Moore Machine

If the output of a sequential network is a function of the present state only, the network is often referred to as a **Moore Machine**.



4.4.2 Mealy Machine

If the output is a function of both the present state and the input, the network is referred to as a **Mealy Machine**.



Chapter 5

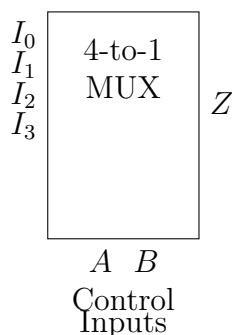
Combinational Logic Components

5.1 Multiplexers

A multiplexer has a group of data inputs and a group of control inputs. The control inputs are used to select one of the data inputs and connect it to the output terminal.

For an n -variable function:

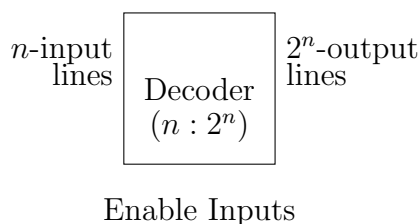
$$Z = A'B'I_0 + A'BI_1 + AB'I_2 + ABI_3$$



A 2^n -to-1 multiplexer can be used to realize any $(n + 1)$ variable function with no added gates. n of the variables are used as control inputs and the remaining variable is used as required on the data inputs.

5.2 Decoders

A decoder is a combinational logic circuit that converts binary information from n input lines to a maximum of 2^n unique output lines.



5.3 Encoders

An encoder is a combinational logic circuit. It is the reverse of a decoder function. It has 2^n (or fewer) input lines and n output lines.

5.4 Canonical Logic Forms

Each term in SOP and POS forms contains all the literals. These forms are called **Canonical** (Standard) SOP and POS expressions.

5.4.1 Half Adder

$$S = A \oplus B \quad C = AB$$

5.4.2 Full Adder

$$S = A \oplus B \oplus C_{in} \quad C_{out} = (A \oplus B)C_{in} + AB$$

5.5 Look-Ahead Carry Adder

For a 4-bit adder:

$$\begin{aligned} S_i &= P_i \oplus C_i & G_i &= A_i \cdot B_i \\ C_{i+1} &= G_i + P_i C_i \end{aligned}$$

where $i = 0, 1, 2, 3$.

$$\begin{aligned} C_1 &= G_0 + P_0 C_0 \\ C_2 &= G_1 + P_1 C_1 \\ &= G_1 + P_1(G_0 + P_0 C_0) \\ C_3 &= G_2 + P_2 C_2 \\ &= G_2 + P_2(G_1 + P_1(G_0 + P_0 C_0)) \\ C_4 &= G_3 + P_3 C_3 \\ &= G_3 + P_3(G_2 + P_2(G_1 + P_1(G_0 + P_0 C_0))) \end{aligned}$$

Chapter 6

Arithmetic Circuits

6.1 Half Subtractor

$$D = A \oplus B \quad B_{out} = \bar{A}B$$

6.2 Full Subtractor

$$D = A \oplus B \oplus B_{in} \quad B_{out} = (A \oplus B)B_{in} + \bar{A}B$$

Chapter 7

State Machine Minimization

7.1 Reduction of State Tables

The process involves two steps:

- (1) Elimination of redundant states
- (2) Determination of state equivalence using an implication table

7.1.1 Example

Present State	$I = 0$		$I = 1$	
	Q	Z	Q	Z
A	B	0	C	1
B	C	0	A	0
C	A	0	B	1
D	A	1	C	1

Implication Table:

			A
	$B - C$		
C	$A - D$	$B - C$	
D	X	X	
	A	B	

Since $C \neq D$ due to output difference, the next state is different.
From the implication table:

$$A \equiv B \equiv C$$

New State Table:

	$I = 0$		$I = 1$	
A	A	0	B	1
B	A	0	B	1
D	A	1	B	1

Chapter 8

Code Conversion

8.1 Binary Code to Gray Code Conversion

Binary Code:

(1) (0) (1) (1) (0)

Gray Code:

(1) (0) (1) (1) (0)

Same

8.2 Gray Code to Binary Code Conversion

Gray Code:

(1) (0) (1) (1) (0)

Binary Code:

(1) (0) (1) (1) (1)

Same